

SOFTWARE ENGINEERING – WRITING QUESTIONS QUICK TEMPLATE

(Use-case / Component / Class / Deployment)

=====

GENERAL EXAM STRATEGY

=====

1. Read the case → underline ACTORS, FUNCTIONS, DATA.
2. Use ONLY required diagram type.
3. Draw SIMPLE, CLEAR, STANDARD UML (no fancy stuff).
4. If unsure: add reasonable assumptions (acceptable in grading).

=====

A. USE-CASE DIAGRAM (1.5 pts)

=====

STEP 1 – Identify ACTORS

- Human users (User, Customer, Admin, HR Staff)
- External systems (Bank, CRM, Payment Gateway)

STEP 2 – Identify MAIN USE CASES (verbs)

- Login, Register
- Create / Open / Buy
- Pay / Transfer / Use
- View / Search / Query

STEP 3 – Relationships

- Actor — UseCase (association)
- <<include>> : mandatory reused behavior (Login, Authenticate)
- <<extend>> : optional behavior

DEFAULT PATTERN:

Actor → Login

Actor → Main Functions

=====

B. COMPONENT DIAGRAM (1.0 pt)

=====

RULE:

- Each service = ONE component
- Components communicate via API (interface)

STANDARD PATTERN:

[UI / Mobile App]

|

[Login] → [Authentication]

|

[Business Service]

|

[External Service]

WRITE APIs:

- transferInternal()
- transferInterBank()
- buyTicket()
- verifyTicket()

=====

C. CLASS DIAGRAM (1.5 pts)

=====

STEP 1 – Core Entities

- User / Employee
- Account / Ticket / Payment

STEP 2 – Attributes (simple)

- id, balance, type, status

STEP 3 – Methods (key only)

- calculateInterest()
- pay()
- deduct()

STEP 4 – Relationships

- Inheritance: abstract Account → Saving / Loan
- Association: User 1..* Account
- Composition: Employee owns PaymentMethod

EXAM KEYWORDS:

- abstract class
- polymorphism
- monthly calculation

=====

D. DEPLOYMENT DIAGRAM (1.0 pt)

=====

NODES:

- Mobile Device

- Application Server
- External Systems

PATTERN:

[Mobile App]

|

[Backend Server]

|

[External Service]

=====

SAFE ASSUMPTIONS TO WRITE

=====

- All users must authenticate
- Services communicate via REST API
- System follows layered architecture
- Interest calculation handled monthly

=====

TIME MANAGEMENT

=====

- Use-case: 10–12 min
- Component: 5–7 min
- Class diagram: 12–15 min
- Deployment: 5 min

=====

MARKING FRIENDLY TIPS

=====

- Use correct UML notation
- Label relationships
- Do not overcomplicate
- Clean layout > full coverage